

API Formularios

Damián Nicolalde – R.

danicolalde@puce.edu.ec

Temas

- Procesando formularios
- Validación
- Seudoclases

Procesando formularios

- La API Formularios es un grupo de propiedades, métodos y eventos que podemos usar para procesar formularios y crear nuestro propio sistema de validación.
- La API está integrada en los formularios y elementos de formularios, por lo que podemos responder a eventos o llamar a los métodos desde los mismos elementos.

Métodos disponibles para el elemento <form>

- **submit()** —Este método envía el formulario.
- **reset()** —Este método reinicializa el formulario (asigna los valores por defecto a los elementos).
- **checkValidity()** —Este método devuelve un valor booleano que indica si el formulario es válido o no.

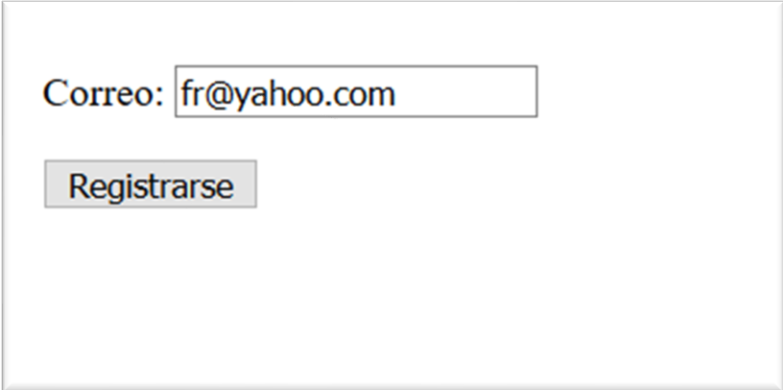
La API también ofrece el siguiente evento para anunciar cada vez que se inserta un carácter o se selecciona un valor en un elemento de formulario.

- **input** —Este evento se desencadena en el formulario o sus elementos cuando el usuario inserta o elimina un carácter en un campo de entrada, y también cuando se selecciona un nuevo valor.
- **change** —Este evento se desencadena en el formulario o sus elementos cuando se introduce o selecciona un nuevo valor.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="utf-8">
5  <title>Formularios</title>
6  <script>
7  function iniciar() {
8  var boton = document.getElementById("enviar");
9  boton.addEventListener("click", enviarformulario);
10 }
11 function enviarformulario() {
12 var formulario = document.querySelector("form[name='informacion']");
13 formulario.submit();
14 }
15 window.addEventListener("load", iniciar);
16 </script>
17 </head>
18 <body>
19   <section>
20     <form name="informacion" method="get" action="procesar.php">
21     <p><label>Correo: <input type="email" name="correo" id="correo"
22     required></label></p>
23     <p><button type="button" id="enviar">Registrarse</button></p>
24     </form>
25   </section>
26 </body>
27 </html>

```



Este ejemplo envía el formulario cuando se pulsa un botón.

- El código responde al evento **click** desde el elemento **<button>** para ejecutar la función **enviarformulario()** cada vez que se pulsa el botón.
- En esta función, obtenemos una referencia al elemento **<form>** y luego enviamos el formulario con el método **submit()**.
- Se ha decidido obtener la referencia al elemento **<form>** con el método **querySelector()** y un selector que busca el elemento por medio de su atributo **name**, pero podríamos haber agregado un atributo **id** al elemento para obtener la referencia con el método **getElementById()**, como hemos hecho con el botón.
- Otra alternativa es obtener la referencia desde la propiedad **forms** del objeto **Document**. Esta propiedad devuelve un array con referencias a todos los elementos **<form>** del documento. En nuestro caso, solo tenemos un elemento **<form>** y, por lo tanto, la referencia se encuentra en el índice 0.

método checkValidity()

- Para que un formulario esté validado, debemos usar el método checkValidity()
- Este método controla el formulario y devuelve true o false para indicar si es válido o no
- Si el formulario es válido, se envía con el método submit(). En caso contrario, se muestra un mensaje en pantalla para advertir al usuario

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="utf-8">
5  <title>Formularios</title>
6  <script>
7  function iniciar() {
8  var boton = document.getElementById("enviar");
9  boton.addEventListener("click", enviarformulario);
10 }
11 function enviarformulario() {
12 var formulario = document.querySelector("form[name='informacion']");
13 var valido = formulario.checkValidity();
14 if (valido) {
15 formulario.submit();
16 } else {
17 alert("El formulario no puede ser enviado");
18 }
19 }
20 window.addEventListener("load", iniciar);
21 </script>
22 </head>
23 <body>
24 <section>
25 <form name="informacion" method="get" action="procesar.php">
26 <p><label>Correo: <input type="email" name="correo" id="correo"
27 required></label></p>
28 <p><button type="button" id="enviar">Registrarse</button></p>
29 </form>
30 </section>
31 </body>
32 </html>
```

The screenshot shows a web form with a label 'Correo:' followed by an empty email input field. Below the input field is a button labeled 'Registrarse'. A white alert box is displayed in the center of the page, containing the text 'El formulario no puede ser enviado'. At the bottom right of the alert box is a button labeled 'Aceptar'.

Validación

- Existen diferentes maneras de validar formularios en HTML.
- Se puede usar campos de entrada del tipo que requieren validación por defecto, como **email**, convertir un campo regular de tipo **text** en un campo requerido con el atributo **required**, o incluso usar tipos especiales como **pattern** para personalizar los requisitos de validación.
- Sin embargo, cuando tenemos que implementar mecanismos de validación más complejos, como comparar dos o más campos o controlar el resultado de una operación, la única opción es la de personalizar el proceso de validación usando la **API Formularios**.

Errores personalizados

- Los navegadores muestran un mensaje de error cuando el usuario intenta enviar un formulario que contiene un campo no válido. Estos son mensajes predefinidos que describen errores conocidos, pero podemos definir mensajes personalizados para establecer nuestros propios requisitos de validación
- Con este fin, los objetos **Element** que representan elementos de formulario incluyen el siguiente método:

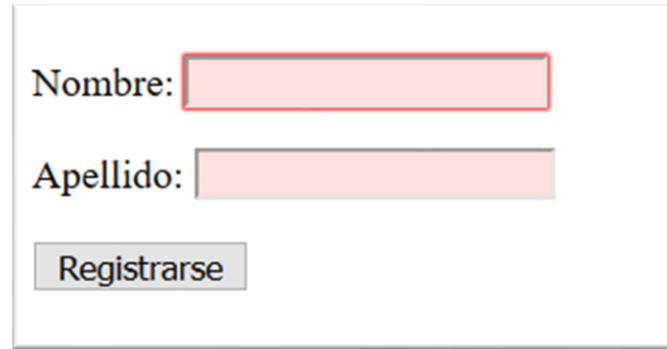
setCustomValidity(mensaje)

Este método declara un error personalizado y el mensaje a mostrar si se envía el formulario. Si no se especifica ningún mensaje, el error se anula

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="utf-8">
5  <title>Formularios</title>
6  <script>
7  var nombre1, nombre2;
8  function iniciar() {
9  nombre1 = document.getElementById("nombre");
10 nombre2 = document.getElementById("apellido");
11 nombre1.addEventListener("input", validacion);
12 nombre2.addEventListener("input", validacion);
13 validacion();
14 }
15 function validacion() {
16 if (nombre1.value == "" && nombre2.value == "") {
17 nombre1.setCustomValidity("Inserte su nombre o su apellido");
18 nombre1.style.background = "#FFDDDD";
19 nombre2.style.background = "#FFDDDD";
20 } else {
21 nombre1.setCustomValidity("");
22 nombre1.style.background = "#FFFFFF";
23 nombre2.style.background = "#FFFFFF";
24 }
25 }
26 window.addEventListener("load", iniciar);
27 </script>
28 </head>
29 <body>
30 <section>
31 <form name="registracion" method="get" action="procesar.php">
32 <p><label>Nombre: <input type="text" name="nombre"
33 id="nombre"></label></p>
34 <p><label>Apellido: <input type="text" name="apellido"
35 id="apellido"></label></p>
36 <p><input type="submit" value="Registrarse"></p>
37 </form>
38 </section>
39 </body>
40 </html>

```



Este ejemplo presenta una situación de validación compleja.

- Se crean dos campos para recibir el nombre y el apellido del usuario.
- Sin embargo, el formulario es no válido cuando ambos campos están vacíos. El usuario puede introducir su nombre o su apellido para validarlo.
- ***En un caso como este, es imposible usar el atributo required porque no sabemos qué campo va a elegir completar el usuario.***
- Solo usando errores personalizados podemos crear un mecanismo de validación efectivo para este escenario.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="utf-8">
5  <title>Formularios</title>
6  <script>
7  var nombre1, nombre2;
8  function iniciar() {
9  nombre1 = document.getElementById("nombre");
10 nombre2 = document.getElementById("apellido");
11 nombre1.addEventListener("input", validacion);
12 nombre2.addEventListener("input", validacion);
13 validacion();
14 }
15 function validacion() {
16 if (nombre1.value == "" && nombre2.value == "") {
17 nombre1.setCustomValidity("Inserte su nombre o su apellido");
18 nombre1.style.background = "#FFDDDD";
19 nombre2.style.background = "#FFDDDD";
20 } else {
21 nombre1.setCustomValidity("");
22 nombre1.style.background = "#FFFFFF";
23 nombre2.style.background = "#FFFFFF";
24 }
25 }
26 window.addEventListener("load", iniciar);
27 </script>
28 </head>
29 <body>
30 <section>
31 <form name="registracion" method="get" action="procesar.php">
32 <p><label>Nombre: <input type="text" name="nombre"
33 id="nombre"></label></p>
34 <p><label>Apellido: <input type="text" name="apellido"
35 id="apellido"></label></p>
36 <p><input type="submit" value="Registrarse"></p>
37 </form>
38 </section>
39 </body>
40 </html>

```

- El código crea referencias a dos elementos **<input>** y agrega listeners al evento **input** para cada uno de ellos. Este evento se desencadena cada vez que el usuario inserta o elimina un carácter, lo que permite detectar cada valor insertado en los campos, y validar o invalidar el formulario desde la **función validacion()**
- Debido a que los elementos **<input>** se encuentran vacíos cuando se carga el documento, tenemos que declarar una condición no válida para no permitir al usuario enviar el formulario antes de insertar al menos uno de los valores en los campos. Por esta razón, la función **validacion()** también se llama al final de la función **iniciar()** para comprobar esta condición.
- La función **validacion()** controla si el formulario es válido o no, y declara o elimina el error con el método **setCustomValidity()**. Si ambos campos están vacíos, se declara un error personalizado y el color de fondo de ambos elementos se cambia a rojo para indicar al usuario el error. Sin embargo, si la condición cambia debido a que se ha introducido al menos uno de los valores, el error se elimina llamando al método con una cadena de caracteres vacía y el color blanco se asigna nuevamente al fondo de ambos campos.

Es importante recordar que el único cambio producido durante el proceso es la modificación del color de fondo de los campos. El mensaje de error que declara el método **setCustomValidity()** solo se mostrará al usuario cuando intente enviar el formulario.

Evento invalid

- Cada vez que el usuario envía un formulario, se desencadena un evento si se detecta un campo no válido.
- El evento se llama **invalid** y se desencadena en el elemento que ha producido el error.
- Para personalizar una respuesta, podemos responder a este evento desde el elemento **<form>**

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="utf-8">
5  <title>Formularios</title>
6  <script>
7  var formulario;
8  function iniciar() {
9  var boton = document.getElementById("enviar");
10 boton.addEventListener("click", enviarformulario);
11 formulario = document.querySelector("form[name='informacion']");
12 formulario.addEventListener("invalid", validacion, true);
13 }
14 function validacion(evento) {
15 var elemento = evento.target;
16 elemento.style.background = "#FFDDDD";
17 }
18 function enviarformulario() {
19 var valido = formulario.checkValidity();
20 if (valido) {
21 formulario.submit();
22 }
23 }
24 window.addEventListener("load", iniciar);
25 </script>
26 </head>
27 <body>
28 <section>
29 <form name="informacion" method="get" action="procesar.php">
30 <p><label>Apodo: <input pattern="[A-Za-z]{3,}" name="apodo"
31 id="apodo" maxlength="10" required></label></p>
32 <p><label>Correo: <input type="email" name="correo" id="correo"
33 required></label></p>
34 <p><button type="button" id="enviar">Registrarse</button></p>
35 </form>
36 </section>
37 </body>
38 </html>

```

- El nuevo formulario tiene dos campos de entrada para introducir un apodo y una cuenta de correo.
- El campo **correo** tiene sus limitaciones naturales debido a su tipo y un atributo **required** que lo declara como campo obligatorio, pero el campo **apodo** contiene tres atributos de validación: el atributo **pattern** que solo admite un mínimo de tres caracteres de la A a la Z (mayúsculas y minúsculas), el atributo **maxlength** que limita el campo a un máximo de diez caracteres, y el atributo **required** que invalida el campo si está vacío.

Apodo:

Correo:

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="utf-8">
5 <title>Formularios</title>
6 <script>
7 var formulario;
8 function iniciar() {
9 var boton = document.getElementById("enviar");
10 boton.addEventListener("click", enviarformulario);
11 formulario = document.querySelector("form[name='informacion']");
12 formulario.addEventListener("invalid", validacion, true);
13 }
14 function validacion(evento) {
15 var elemento = evento.target;
16 elemento.style.background = "#FFDDDD";
17 }
18 function enviarformulario() {
19 var valido = formulario.checkValidity();
20 if (valido) {
21 formulario.submit();
22 }
23 }
24 window.addEventListener("load", iniciar);
25 </script>
26 </head>
27 <body>
28 <section>
29 <form name="informacion" method="get" action="procesar.php">
30 <p><label>Apodo: <input pattern="[A-Za-z]{3,}" name="apodo"
31 id="apodo" maxLength="10" required></label></p>
32 <p><label>Correo: <input type="email" name="correo" id="correo"
33 required></label></p>
34 <p><button type="button" id="enviar">Registrarse</button></p>
35 </form>
36 </section>
37 </body>
38 </html>

```

El código es muy similar a los ejemplos anteriores.

1. Responde al evento **load** con la función **iniciar()** cuando el documento termina de cargarse y al evento **click** del elemento **<button>**, como siempre
2. Luego agregamos un listener para el evento **invalid** al elemento **<form>** en lugar de los campos **<input>**. Esto se debe a que queremos establecer un sistema de validación para todo el formulario, no solo elementos individuales. Para este propósito, tenemos que incluir el valor **true** como tercer atributo del método **addEventListener()**.
 - Este atributo le indica al navegador que tiene que propagar el evento al resto de los elementos de la jerarquía.
 - Como resultado, a pesar de que el *listener* se ha agregado al elemento **<form>**, este responde a eventos desencadenados por los elementos dentro del formulario. Para determinar cuál es el elemento no válido que ha llamado a la función **validacion()**, leemos el valor de la propiedad **target**.
 - Usando esta referencia, la última instrucción en esta función cambia el color de fondo del elemento a rojo.

Cuando se carga el formulario

Apodo:

Correo:

Cuando se envía el formulario sin valores

Apodo:

Correo:

El objeto `ValidityState`

- El ejemplo anterior no realiza una validación en tiempo real. Los campos se validan solo cuando se envía el formulario.
- Considerando la necesidad de un sistema de validación más dinámico, la API Formularios incluye el **objeto `ValidityState`**.
- Este objeto ofrece una serie de propiedades para indicar el estado de validez de un elemento del formulario.

`valid`—Esta propiedad devuelve `true` si el valor del elemento es válido.

- La propiedad **`valid`** devuelve el estado de validez de un elemento considerando todos los demás estados de validez. Si todas las condiciones son válidas, la propiedad **`valid`** devuelve **`true`**.
- El objeto `ValidityState` se asigna a una propiedad llamada `validity`, disponible en cada elemento de formulario

Propiedades del objeto ValidityState

- **valueMissing**—Esta propiedad devuelve **true** cuando se ha declarado el atributo **required** y el campo está vacío
- **typeMismatch**—Esta propiedad devuelve **true** cuando la sintaxis del texto introducido no coincide con el tipo de campo. Por ejemplo, cuando el texto introducido en un campo de tipo email no es una cuenta de correo
- **patternMismatch**—Esta propiedad devuelve **true** cuando el texto introducido no respeta el formato establecido por el atributo **pattern**
- **tooLong**—Esta propiedad devuelve **true** cuando se ha declarado el atributo **maxlength** y el texto introducido es más largo que el valor especificado por este atributo
- **rangeUnderflow**—Esta propiedad devuelve **true** cuando se ha declarado el atributo **min** y el valor introducido es menor que el especificado por este atributo
- **rangeOverflow**—Esta propiedad devuelve **true** cuando se ha declarado el atributo **max** y el valor introducido es mayor que el especificado por este atributo
- **stepMismatch**—Esta propiedad devuelve **true** cuando se ha declarado el atributo **step** y el valor introducido no corresponde con el valor de los atributos **min**, **max** y **value**
- **customError**—Esta propiedad devuelve **true** cuando declaramos un error personalizado con el método **setCustomValidity()**

Validación en tiempo real

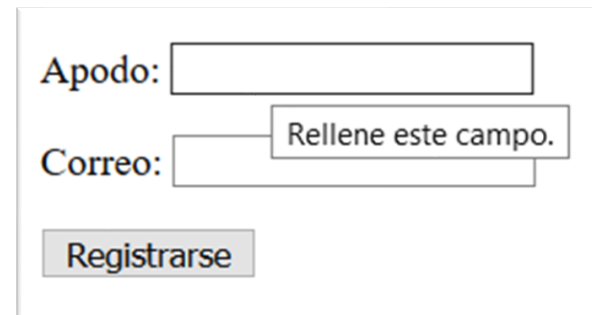
- Al código del ejemplo anterior, agregamos un listener para el evento input al formulario.
- Cada vez que el usuario modifica un campo, insertando o eliminando un carácter, se ejecuta la función **comprobar()** para responder al evento.
- La función **comprobar()** también aprovecha la propiedad target para obtener una referencia al elemento que desencadenó el evento y controlar su validez leyendo el valor de la propiedad **valid** dentro de la propiedad **validity** del objeto **Element** (**elemento.validity.valid**)
- Con esta información, cambiamos el color de fondo del elemento que desencadenó el evento **input** en tiempo real. El color será rojo hasta que el texto introducido por el usuario sea válido.

```
3 <head>
4 <meta charset="utf-8">
5 <title>Formularios</title>
6 <script>
7 var formulario;
8 function iniciar() {
9   var boton = document.getElementById("enviar");
10  boton.addEventListener("click", enviarformulario);
11  formulario = document.querySelector("form[name='informacion']");
12  formulario.addEventListener("invalid", validacion, true);
13  formulario.addEventListener("input", comprobar);
14 }
15 function validacion(evento) {
16   var elemento = evento.target;
17   elemento.style.background = "#FFDDDD";
18 }
19 function enviarformulario() {
20   var valido = formulario.checkValidity();
21   if (valido) {
22     formulario.submit();
23   }
24 }
25 function comprobar(evento) {
26   var elemento = evento.target;
27   if (elemento.validity.valid) {
28     elemento.style.background = "#FFFFFF";
29   } else {
30     elemento.style.background = "#FFDDDD";
31   }
32 }
33 window.addEventListener("load", iniciar);
34 </script>
35 </head>
```

Validación en tiempo real

- Se puede usar el resto de las propiedades que ofrece el objeto `ValidityState` para saber exactamente qué ha producido el error.
- Al modificar la función **`enviarformulario()`** para detectar errores específicos. El formulario valida el método **`checkValidity()`** y, si es válido, se envía con el método **`submit()`**. En caso contrario, se leen los valores de las propiedades **`patternMismatch`** y **`valueMissing`** del campo `apodo` y se muestra un mensaje de error cuando una o ambas devuelven **`true`**.

Al pasar el ratón sobre el campo

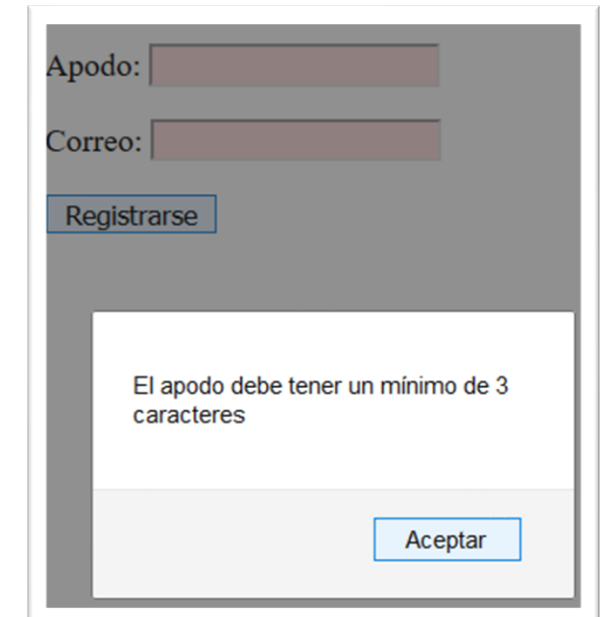


Apodo:

Correo:

Registrarse

Al enviar sin datos



Apodo:

Correo:

Registrarse

El apodo debe tener un mínimo de 3 caracteres

Aceptar

```
6 <script>
7 var formulario;
8 function iniciar() {
9   var boton = document.getElementById("enviar");
10  boton.addEventListener("click", enviarformulario);
11  formulario = document.querySelector("form[name='informacion']");
12  formulario.addEventListener("invalid", validacion, true);
13  formulario.addEventListener("input", comprobar);
14 }
15 function validacion(evento) {
16   var elemento = evento.target;
17   elemento.style.background = "#FFDDDD";
18 }
19 function enviarformulario() {
20   var elemento = document.getElementById("apodo");
21   var valido = formulario.checkValidity();
22   if (valido) {
23     formulario.submit();
24   } else if (elemento.validity.patternMismatch ||
25             elemento.validity.valueMissing) {
26     alert('El apodo debe tener un mínimo de 3 caracteres');
27   }
28 }
29 function comprobar(evento) {
30   var elemento = evento.target;
31   if (elemento.validity.valid) {
32     elemento.style.background = "#FFFFFF";
33   } else {
34     elemento.style.background = "#FFDDDD";
35   }
36 }
37 window.addEventListener("load", iniciar);
38 </script>
```

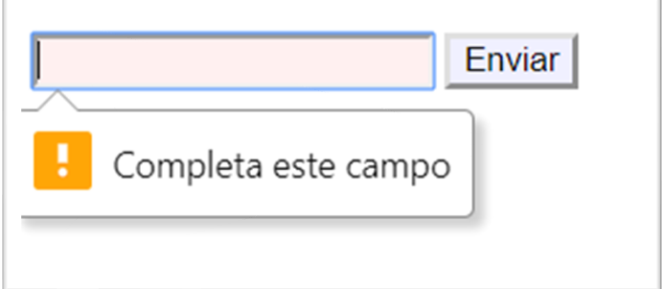
Seudoclases

- Además de todas las propiedades y métodos provistos por la API Formularios, CSS incluye algunas seudoclases para modificar los estilos de un elemento dependiendo de su estado, incluidos inválido, válido, requerido, opcional, e incluso cuando un valor se encuentra fuera del rango permitido

Valid e Invalid

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="utf-8">
5 <title>Formularios</title>
6 <style>
7 input:valid{
8     background: #EEEEFF;
9 }
10 input:invalid{
11     background: #FFEEEE;
12 }
13 </style>
14 </head>
15 <body>
16 <section>
17 <form name="formulario" method="get" action="procesar.php">
18 <input type="email" name="correo" required>
19 <input type="submit" value="Enviar">
20 </form>
21 </section>
22 </body>
23 </html>
```

- Estas pseudoclases afectan a cualquier elemento **<input>** con un valor válido o inválido.
- El formulario incluye un elemento **<input>** para cuentas de correo.
- Cuando el contenido del elemento no es válido, la pseudoclase **:valid** asigna un color de fondo azul al campo, pero tan pronto como el contenido se vuelve no válido, la pseudoclase **:invalid** cambia el color de fondo a rojo.



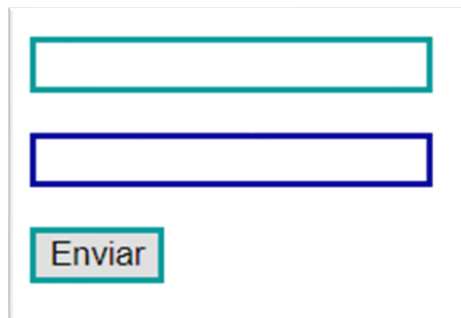
A screenshot of a web form. It features a text input field with a light blue border and a light pink background. To the right of the input field is a button labeled "Enviar". Below the input field, there is a yellow warning icon (an exclamation mark inside a square) followed by the text "Completa este campo".

Optional y Required

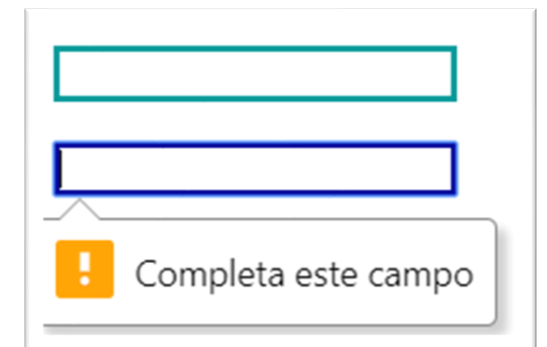
```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="utf-8">
5 <title>Formularios</title>
6 <style>
7   input:optional{
8     border: 2px solid #009999;
9   }
10  input:required{
11    border: 2px solid #000099;
12  }
13 </style>
14 </head>
15 <body>
16   <section>
17     <form name="formulario" method="get" action="procesar.php">
18       <p><input type="text" name="nombre"></p>
19       <p><input type="text" name="apellido" required></p>
20       <p><input type="submit" value="Enviar"></p>
21     </form>
22   </section>
23 </body>
24 </html>
```

- Estas pseudoclases afectan a todos los elementos de formulario declarados como obligatorios u opcionales.
- El ejemplo incluye dos campos de entrada: **nombre** y **apellido**. El primero es opcional, pero apellido es obligatorio. Las pseudoclases asignan un color de borde diferente a estos campos de acuerdo con su condición (el campo obligatorio se muestra en azul y el campo opcional en verde).

Al cargar la página

El formulario se muestra al cargar la página. Tiene dos campos de texto y un botón de envío. El primer campo de texto, correspondiente al 'nombre', tiene un borde verde. El segundo campo de texto, correspondiente al 'apellido', tiene un borde azul. El botón 'Enviar' también tiene un borde verde.

Al enviar sin datos

El formulario se muestra al enviar sin datos. El campo de texto 'apellido' (obligatorio) tiene un borde azul y una alerta que dice 'Completa este campo' con un icono de exclamación naranja. El campo de texto 'nombre' (opcional) sigue teniendo un borde verde. El botón 'Enviar' también tiene un borde verde.

In-range y Out-of-range

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>Formularios</title>
6   <style>
7     input:in-range{
8       background: #EEEEFF;
9     }
10    input:out-of-range{
11      background: #FFEEEE;
12    }
13  </style>
14 </head>
15 <body>
16   <section>
17     <form name="formulario" method="get" action="procesar.php">
18       <input type="number" name="numero" min="0" max="10">
19       <input type="submit" value="Enviar">
20     </form>
21   </section>
22 </body>
23 </html>
```

- Estas pseudoclases afectan a todos los elementos con un valor dentro o fuera de un rango específico.
- En este ejemplo se ha incluido un campo de entrada de tipo number para probar estas pseudoclases.
- Cuando el valor introducido en el elemento es menor que 0 o mayor que 10, el color de fondo es rojo, pero tan pronto como introducimos un valor dentro del rango especificado, el color de fondo cambia a azul.

8	Enviar	23	Enviar
0	Enviar	-1	Enviar